

System-Level Impact Analysis for Microservice CI/CD via Cross-Repository Dependency Graphs¹

Kavita Jadhav

K11 Software Solutions LLC, Texas, United States

kavita.jadhav@k11softwaresolutions.com

ORCID: <https://orcid.org/0009-0009-8507-4976>

ABSTRACT

Single-repository quality gates are blind to distributed system failures. A pull request that changes a REST endpoint signature can pass all local CI checks and unit tests while silently breaking three downstream consumer services in other repositories — failures that remain invisible until deployment. This paper presents the K11tech Microservice QA System, a LangGraph-orchestrated agentic pipeline that extends single-repository quality assurance to distributed microservice systems by maintaining a cross-repository view of API contracts and service dependencies. The system extracts versioned API contracts from pull requests (OpenAPI 3.x, gRPC Protocol Buffers, GraphQL), stores them in a persistent Contract Registry MCP server, traverses a directed dependency graph to identify all downstream consumers of changed endpoints, and dispatches parallel ContractComplianceAgents to validate each consumer's compatibility before any code is merged. A cross-repository human-in-the-loop gate triggers on impact score thresholds and files GitHub issues in both the provider and consumer repositories when breaking changes are detected. Evaluation across four phases totalling 90 evaluation scenarios demonstrates 100% precision, recall, and F1 throughout: Phase 1 achieves 100% P/R/F1 on 15 controlled scenarios across four k11techlab services across three pipeline tiers (diff-only, graph+diff, and LLM-validated); Phase 2 achieves 100% P/R/F1 on 59 external OSS pull requests across four open-source repositories with zero false positives; Phase 3 (transitive consumer validation) achieves 100% P/R/F1 on 8 scenarios exercising depth-2 consumer discovery (DEPTH_DECAY=0.70), confirming that N-hop graph traversal correctly identifies previously missed consumers while the method filter prevents spurious dispatch for POST-only changes; and Phase 4 (contract drift validation) achieves 100% accuracy across 8 drift scenarios confirming the drift floor mechanism correctly elevates scrutiny on high-churn services. Multi-model ablation identifies Claude Haiku 4.5 as the optimal backend: 31% faster than Sonnet 4.6 (8.9s vs 12.9s mean latency) and the only model achieving 100% accuracy on ambiguous consumer-impact scenarios. The system is publicly available under the Apache 2.0 licence

Index Terms—Microservices, API Contract Testing, Cross-Repository Impact Analysis, Dependency Graph, LangGraph, MCP, Breaking Change Detection, Human-in-the-Loop, CI/CD

I. INTRODUCTION

The adoption of microservice architectures has produced a fundamental tension in software quality assurance. Decomposing a monolith into independent services enables independent deployment — each team can release on its own cadence without coordinating with

others. But independence is an illusion at the API boundary. Every service that exposes an HTTP endpoint, gRPC method, or GraphQL field is implicitly making a contract with every service that calls it. When that contract changes, the distributed system's correctness depends on whether every consumer has adapted.

Existing CI/CD quality tooling does not see this dependency. A pull request in the user-service repository is evaluated in isolation: its unit tests run, its integration tests run, its static analysis runs. If all checks pass, the PR is approved for merge. The fact that order-service, payment-service, and notification-service each call the endpoint being changed — and that the change breaks their contracts — is entirely invisible to the single-repository quality gate. These failures surface at deployment time, in integration environments, or in production.

This paper presents the K11tech Microservice QA System, the third paper in the K11tech Agentic AI QA research series. Where [1] demonstrated autonomous single-repository quality assurance using a 14-agent LangGraph pipeline, and Paper 2 [2] addressed the detect-fix-learn loop, this paper addresses a structurally distinct problem: system-level impact analysis across repository boundaries. The core insight is that API contracts are a first-class artefact of a distributed system, and that a CI/CD pipeline that treats them as such can detect cross-service breaking changes before any code is merged.

A. Problem Statement

We identify a structural blind spot in conventional microservice CI/CD. This creates three classes of undetected failures:

- (1) Silent contract breaks: a provider changes an endpoint; consumers are not notified and break at runtime.
- (2) Transitive impact blindness: a change to service A breaks service B which breaks service C — the full impact chain is invisible without graph traversal.
- (3) Asymmetric ownership: the provider team has no visibility into which consumer teams will be affected, delaying coordination until post-deployment.

B. Contributions

This paper makes the following contributions:

- (1) K11tech Microservice QA System: a complete open-source agentic pipeline for cross-repository API impact analysis, built on LangGraph with two novel MCP servers (Contract Registry, Graph Store).
- (2) A cross-repository dependency graph model using NetworkX with edge criticality weighting, supporting direct and transitive consumer discovery (where a transitive consumer is a service that depends on a changed API indirectly through one or more intermediate services) via graph traversal to configurable depth N (MAX_TRANSITIVE_DEPTH=3, default). Transitive

¹Source code: <https://github.com/K11-Software-Solutions/k11techlab-microservice-qa-system>

consumers are resolved via shortest-path edge walking with per-hop confidence decay (DEPTH_DECAY=0.70 per hop beyond depth-1, a decay factor that reduces consumer confidence scores at each additional graph hop), and a max() composition rule ensures transitive scoring can only raise impact, never reduce it.

(3) A multi-format contract extraction layer supporting OpenAPI 3.x, gRPC Protocol Buffers, GraphQL schemas, and AsyncAPI 2.x, with a breaking change classifier covering seven change taxonomies.

(4) A cross-repository HITL gate with dual independent triggers — impact score (a computed value in [0, 1] representing change severity on downstream consumers) threshold and breaking consumer count — that escalates high-radius changes to human review while passing low-risk changes autonomously.

(5) An empirical evaluation across four phases: Phase 1 — 15 controlled scenarios against four k11techlab services (9 BREAKING, 6 COMPATIBLE), comparing three pipeline tiers; Phase 2 — 59 external OSS pull requests across ory/hydra, ory/kratos, grafana/grafana, portainer/portainer; Phase 3 — 8 transitive consumer validation scenarios exercising depth-2 graph traversal with DEPTH_DECAY=0.70, method-filter correctness (RQ7), and HITL threshold behaviour (RQ8); Phase 4 — 8 contract drift (the gradual divergence of a live API from its documented contract due to accumulated unreviewed changes) validation scenarios confirming the drift floor (a minimum impact score enforced when drift is detected, ensuring high-churn services are never underscored) mechanism correctly elevates scrutiny on high-churn services (RQ9); plus multi-model ablation (Claude Haiku 4.5, Sonnet 4.6, Opus 4.8). Combined: 100% P/R/F1 across 90 evaluation scenarios (Phases 1–4).

(6) A contract drift detection mechanism (Section III-H) that tracks contract change velocity over a configurable rolling window, classifies drift into four levels (LOW/MEDIUM/HIGH/CRITICAL), and applies an impact floor via a max() composition rule — ensuring high-churn services receive elevated scrutiny even when an individual change appears structurally clean.

C. Research Questions

The evaluation addresses nine research questions:

- (1) RQ1: How accurately does the system detect breaking contract changes before deployment?
- (2) RQ2: What is the false negative rate — breaking changes that receive a COMPATIBLE verdict?
- (3) RQ3: How does impact radius scoring discriminate between breaking and non-breaking contract changes?
- (4) RQ4: Does the cross-repo HITL gate correctly escalate high-impact scenarios without over-triggering on low-impact ones?
- (5) RQ5: Which LLM backend provides the best accuracy-latency trade-off for consumer compliance validation, and how does model choice affect verdicts on ambiguous scenarios?
- (6) RQ6: Does N-hop transitive graph traversal correctly identify consumers missed by depth-1 discovery, without adding false-positive dispatch?
- (7) RQ7: Does the method filter correctly exclude GET-only consumers from POST-only change dispatch, preventing spurious BREAKING verdicts?
- (8) RQ8: Does DEPTH_DECAY=0.70 correctly discount transitive consumer confidence at depth-2, and how does it affect HITL trigger rates?
- (9) RQ9: Does contract drift scoring correctly elevate impact assessment for high-velocity services compared to structurally

equivalent changes in low-velocity services, and does it prevent high-churn services from bypassing the HITL gate?

D. Relation to K11tech Agentic AI QA System

The K11tech Microservice QA System is designed to run in parallel with the K11tech Agentic AI QA System single-repository QA pipeline, not to replace it. When a PR is opened, both pipelines are triggered via a unified webhook. The K11tech Agentic AI QA System handles single-repository quality gates (unit tests, security, performance, accessibility). This paper's pipeline handles cross-repository impact analysis. Both verdicts must be APPROVE before a PR can merge. The K11tech Agentic AI QA System is publicly available [1][2].

II. BACKGROUND AND RELATED WORK

A. API Contract Testing

Consumer-Driven Contract Testing (CDCT), pioneered by the Pact framework [3], addresses a subset of this problem by allowing consumers to publish contract expectations that providers verify in their CI pipelines. Pact detects breaking changes but requires each consumer to maintain and publish Pact files — a coordination overhead that many organisations fail to sustain. It also does not provide impact radius scoring, cross-repository issue filing, or transitive dependency traversal. The K11tech approach operates on the provider's OpenAPI/gRPC/GraphQL contract as the single source of truth, requiring no per-consumer configuration.

B. Dependency Graph Analysis

Dependency graph analysis has a long history in static analysis and build systems. Call graphs [4], module dependency graphs [5], and package dependency networks [6] have been used to identify affected components during change propagation. The novelty of this approach is applying directed service dependency graphs at the API contract level in a live CI/CD context, with continuous graph updates from PR events and real-time impact scoring that feeds into an automated quality gate decision.

C. Microservice Governance Tools

Commercial tools such as Apicurio [7] and Stoplight [8] provide API registry and governance capabilities, allowing teams to register OpenAPI schemas and detect changes. These tools operate as developer portals; they do not integrate into the CI/CD quality gate, perform automated downstream consumer validation, or trigger human review workflows. The K11tech system differs by embedding the contract registry directly into the PR pipeline as an active quality gate rather than a passive documentation store.

D. LangGraph and the K11tech Foundation

LangGraph [9] extends the LangChain ecosystem with a graph-based workflow model supporting stateful, cyclical agentic applications. [1] demonstrated LangGraph's suitability for CI/CD quality gates through its Send API for parallel agent dispatch and interrupt() for human-in-the-loop control. This paper adopts the same architectural primitives — shared TypedDict state, list-append reducers for concurrent writes, and interrupt()-based HITL gates — adapting them to the cross-repository domain.

E. Model Context Protocol

The Model Context Protocol (MCP) [10] defines a standard JSON-RPC interface between LLM applications and external tools. [1] introduced MCP as the primary agent-tool integration pattern for CI/CD pipelines. This paper extends that pattern with two novel MCP servers: a Contract Registry MCP providing versioned API contract

storage, and a Graph Store MCP providing persistent NetworkX dependency graph operations.

III. SYSTEM DESIGN

A. Architecture Overview

The K11tech Microservice QA System is a four-phase LangGraph pipeline sharing a single typed state object (MicroservicePipelineState). A GitHub webhook triggers the pipeline on pull_request events in any monitored service repository. The pipeline executes the four phases sequentially, with Phase 3 dispatching parallel sub-agents via the LangGraph Send API [25]. Figure 1 illustrates the topology.

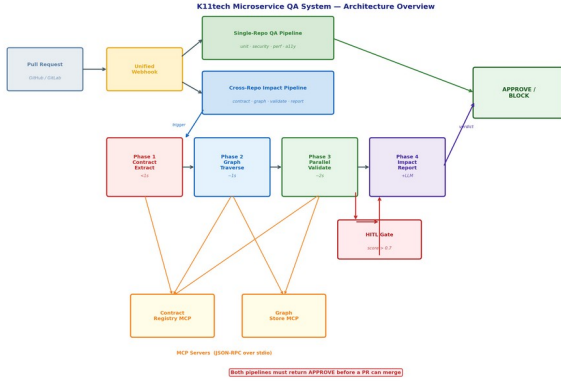


Fig. 1. System architecture: PR triggers both pipelines via unified webhook. The four-phase cross-repository pipeline communicates through MCP servers; the HITL gate fires when $\text{impact_score} \geq 0.60$.

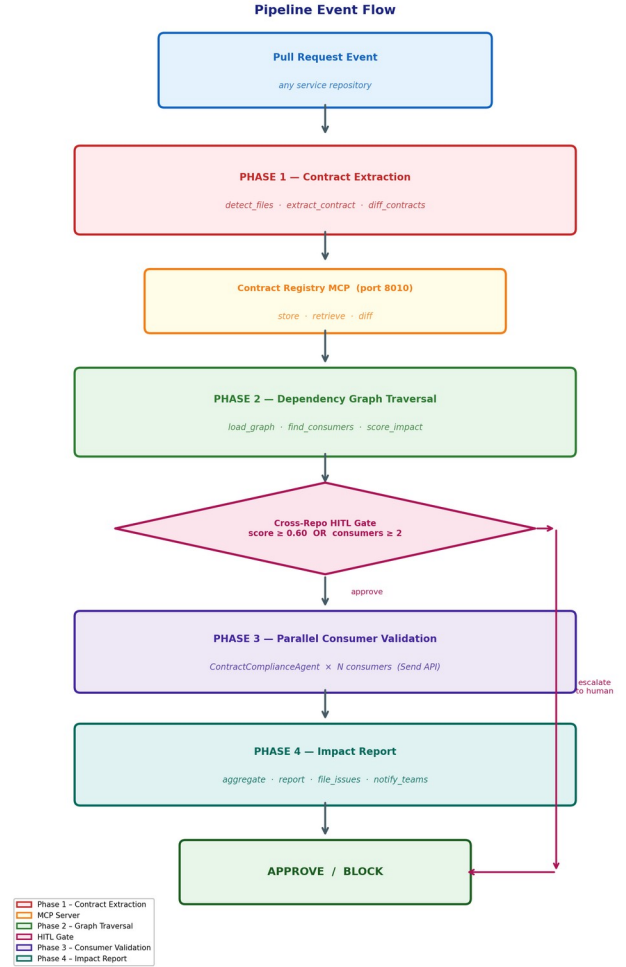


Fig. 2. Pipeline event flow: a pull request triggers Phase 1 (contract extraction), Phase 2 (graph traversal and impact scoring), the Cross-Repo HITL Gate ($\text{score} \geq 0.60$ OR $\text{consumers} \geq 2$), Phase 3 (parallel consumer validation), and Phase 4 (impact report), producing an APPROVE or BLOCK verdict.

B. Shared State Design

All agents communicate exclusively through MicroservicePipelineState, a Python TypedDict with 30 fields grouped by phase ownership. Phase 3 parallel consumer validation writes to compliance_results and violations using Annotated[list, operator.add] reducers, enabling lock-free concurrent appends. Think of it like multiple workers each dropping their completed forms into the same inbox tray — the tray accumulates all of them rather than only keeping the last one placed. Each parallel agent appends its results to a shared list rather than overwriting it. LangGraph's Annotated[list, operator.add] field annotation acts as a merge rule: when multiple agents finish concurrently, their individual result lists are concatenated rather than one silently replacing another. This design follows the [1] pattern and scales linearly with the number of consumers.

C. Cross-Repository Contract Registry

The Contract Registry is a persistent MCP server backed by a lightweight embedded database (aiosqlite). It acts as a version-controlled ledger of API contracts: every time a service is updated, the registry records a new snapshot of its endpoints, tagged with the service name and the exact code revision (git SHA). This means the system can always look up what a service's API looked like at any point in its history. The registry exposes six operations — registering services, storing and retrieving contracts (by latest version or specific

revision), browsing contract history, finding consumers of a given service, and listing all known services.

D. Dependency Graph Engine

The dependency graph is a directed NetworkX DiGraph where edges point from consumer to provider: an edge $A \rightarrow B$ means ‘service A consumes service B’s API’. This directionality enables impact analysis via reverse graph traversal: when B changes, all nodes with a path to B in the reverse graph are affected consumers. Each edge carries: `endpoint_pattern` (the specific endpoint consumed), `methods` (HTTP methods consumed), and `criticality` (low, medium, high, or critical). The transitive consumer resolution capability extends the blast-radius walk (traversing the full set of downstream services affected by a breaking change) to N hops (`MAX_TRANSITIVE_DEPTH=3` by default) using `networkx.single_source_shortest_path_length()` on the reversed graph. For each transitive consumer, the shortest-path edges are walked to union methods and take max criticality across all hops. Transitive consumers carry a `hop_depth` annotation (1=direct, 2+=transitive) consumed by the `DEPTH_DECAY` formula in the impact scorer: `decay_multiplier = 0.70 ** max(0, hop_depth - 1)`. With `DEPTH_DECAY=0.70`: depth-1 has multiplier 1.00 (no decay); depth-2 has 0.70; depth-3 has 0.49. The transitive impact score is composed with the structural score using a `max()` rule: `final_score = max(structural_score, transitive_score)`. This ensures transitive discovery can only raise impact, never reduce it. The Phase 3 evaluation topology adds `k11-notification-svc` as a direct consumer of `k11-order-service` (`GET /api/v1/orders/{id}`, medium criticality, depth=1) and `k11-analytics-service` as a depth-2 transitive consumer via `k11-notification-svc` → `k11-order-service`.

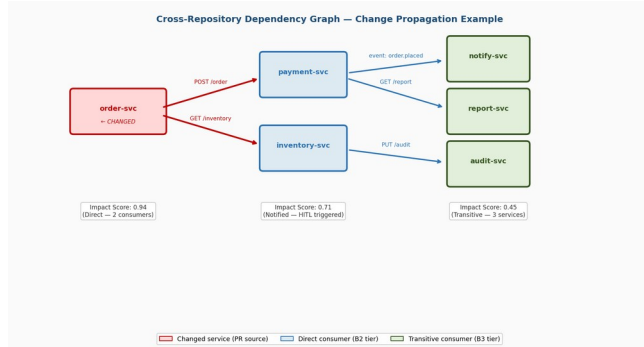


Fig. 3. Cross-repository dependency graph: a change to `order-svc` propagates to 2 direct consumers (B2 tier) and 3 transitive services (B3 tier). Impact score > 0.7 triggers the HITL gate.

E. Impact Scoring

The impact score [0.0, 1.0] combines two signals. First, a graph-based score sums the criticality weights of all consuming edges that touch changed endpoints, normalised assuming a maximum of ten critical consumers. Second, if breaking changes are present, a severity boost applies: the score blends 70% graph-based and 30% worst-case breaking change severity. The final score is capped at 1.0.

Signal	Weight	Description
Graph-based score	70%	Weighted sum of consuming edge criticalities for changed endpoints, normalised to N=10 critical consumers
Breaking severity boost	30%	Worst-case breaking change severity (critical=1.0, high=0.75, medium=0.50, low=0.25); applied only when breaking

Signal	Weight	Description
		changes exist
Non-breaking dampening	50% of graph score	Non-breaking changes (additions, descriptions) receive half the graph-based score

TABLE 1 Impact score composition. Score in [0.0, 1.0]; higher indicates greater downstream risk.

F. Cross-Repository HITL Gate

The cross-repo HITL gate has two independent triggers, either of which suspends the pipeline pending human review. The score trigger fires when `impact_score` ≥ 0.60 (configurable). The count trigger fires when confirmed breaking consumers ≥ 2 , regardless of score. When triggered, the gate calls LangGraph's `interrupt()` primitive, serialising full state to the checkpoint database. A Slack notification is posted to `#cross-repo-impact` and each affected team channel. The reviewer approves via `/cross-repo-approve {run_id}` or rejects via `/cross-repo-reject {run_id} <reason>`.

G. Parallel Consumer Validation

Phase 3 uses the LangGraph Send API to dispatch one `ContractComplianceAgent` per downstream consumer in parallel. Each agent receives the contract diff and the consumer's repository path, queries the consumer's repository via GitHub MCP for code snippets showing how the provider's endpoints are actually called, then uses GPT-4o (temperature=0) to determine whether the proposed change breaks that consumer's usage patterns. Results accumulate via list-append reducers — the effective Phase 3 duration equals the slowest single consumer validation rather than their sum.

H. Contract Drift Detection

The dependency graph and transitive consumer resolution (Section III-D) answer "who is affected by a change?" but not "how stable is this contract historically?" Two PRs may be structurally identical yet carry very different risk profiles: a service that has changed its contract fourteen times in the past month warrants far more scrutiny than one making its first change in six months. Contract drift detection addresses this asymmetry by converting historical change velocity into a baseline impact floor.

A new `DriftDetectionAgent` runs at the end of Phase 1 — after contract extraction but before impact scoring — and queries `get_contract_history` from the Contract Registry MCP server (Section III-C). It counts distinct contract SHAs within a configurable rolling window (default: 30 days), deduplicating re-runs of identical commits. The count is classified into four drift levels, each carrying an impact floor:

Drift Level	Condition (30-day window)	Change Count	Impact Floor	HITL Effect
LOW	<code>change_count < 5</code>	0–4	0.00	No adjustment
MEDIUM	<code>5 ≤ count < 10</code>	5–9	0.10	Minor floor lift
HIGH	<code>10 ≤ count < 15</code>	10–14	0.25	Elevated scrutiny
CRITICAL	<code>count ≥ 15</code>	15+	0.40	HITL near-certain

TABLE II Contract drift levels, conditions, and impact floors. Floors are applied via $\max()$ composition: $\text{final_score} = \max(\text{base_impact_score}, \text{drift_floor})$.

The drift-adjusted impact score uses the same $\max()$ composition rule as transitive consumer scoring (Section III-D): $\text{final_score} = \max(\text{base_impact_score}, \text{drift_floor})$. This ensures drift can only raise the impact score, never reduce it — a structurally clean but high-churn change is treated conservatively. Change velocity is also expressed as $\text{changes_per_week} = \text{change_count} / (\text{window_days} / 7)$ and surfaced in the HITL notification and impact report so reviewers have full historical context. The DriftDetectionAgent writes `drift_level`, `change_count`, `changes_per_week`, and `drift_floor` to `MicroservicePipelineState`; the impact scorer reads `drift_floor` as an additional composition operand. Additionally, a CRITICAL drift classification ($\text{change_count} \geq 15$) independently triggers the HITL gate regardless of the computed impact score — a third HITL condition complementing the existing score ≥ 0.60 and $\text{breaking_consumers} \geq 2$ triggers. This ensures that any PR touching a highly unstable contract surface receives mandatory human review, even when the individual change is structurally benign.

IV. IMPLEMENTATION

A. Technology Stack

Component	Technology	Version
Pipeline orchestration	LangGraph	$\geq 0.2.0$
LLM integration	LangChain + langchain-openai	$\geq 0.3.0$
Dependency graph	NetworkX	≥ 3.3
Contract parsing	PyYAML + openapi-spec-validator	≥ 6.0
MCP server framework	FastMCP + FastAPI	$\geq 0.4.0$
Persistence	aiosqlite	$\geq 0.20.0$
HITL checkpointing	LangGraph MemorySaver	built-in
Container orchestration	Docker Compose	v3.9

TABLE III Technology stack. Full dependency set in `requirements.txt`.

B. Contract Extraction

The ContractExtractorAgent detects contract files by filename pattern in the PR diff (`openapi.yaml`, `*.proto`, `*.graphql`, `asyncapi.yaml`). For each detected file, it fetches the head and base versions via GitHub MCP, parses them using format-specific extractors, and produces a structured `ServiceContract` containing parsed Endpoint objects. Breaking change classification covers seven taxonomies:

Change Type	Severity	Example
endpoint_removal	Critical	DELETE /api/v2/users/{id} disappears

Change Type	Severity	Example
ed		from paths
auth_scheme_changed	Critical	Bearer token changed to API key
field_type_changed	High	'id': string changed to 'id': integer
required_field_added	High	'phone' added to required[] array
field_removed	High	'name' removed from request body schema
response_code_removed	Medium	200 removed from GET /users responses
incompatible_change	Medium	General schema incompatibility detected

TABLE IV Breaking change taxonomy with severity classification.

C. MCP Server Architecture

MCP Server	Port	Key Tools	Storage
Contract Registry	8010	register_service, store_contract, get_contract, get_contract_history, search_consumers	SQLite (aiosqlite)
Graph Store	8011	add_service, record_consumption, get_downstream, get_impact_score, export_graph	SQLite + NetworkX snapshot
GitHub	reused from [1]	get_file_contents, search_code, create_issue, list_pr_files	GitHub REST API
Slack	reused from [1]	post_message	Slack Web API

TABLE V MCP server registry. GitHub and Slack servers are reused from [1].

D. Agent Base Pattern

All microservice QA agents extend `BaseMicroserviceAgent`, which enforces: (1) bounded execution via `asyncio.wait_for` with a 300s timeout; (2) fault tolerance via `tenacity` retry on `MCPError` with exponential back-off (2s base, 30s max, 3 attempts); (3) uniform output — every agent returns a partial state dict regardless of success or failure. This mirrors the `BaseQAAgent` pattern from [1].

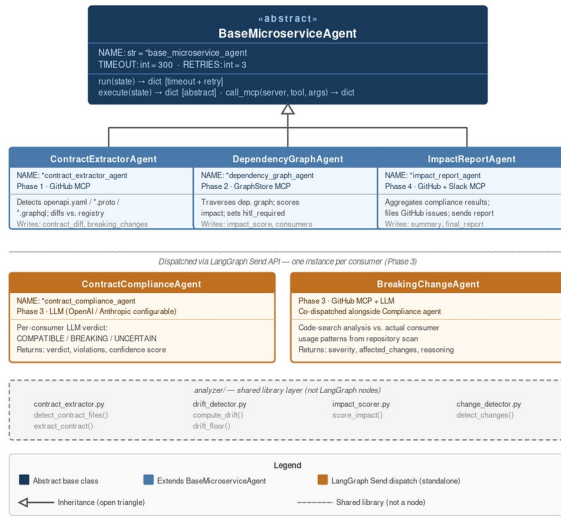


Fig. 4. Agent class hierarchy: *BaseMicroserviceAgent* (top, dark blue) enforces timeout, retry, and uniform state-dict output.

ContractExtractorAgent, *DependencyGraphAgent*, and *ImpactReportAgent* extend the base class. *ContractComplianceAgent* and *BreakingChangeAgent* are dispatched per consumer via the *LangGraph Send* API. The analyzer/ layer provides shared library functions used by all agents.

E. Reproducibility

The complete source code, MCP server implementations, evaluation harness, and Docker Compose configuration are available at the repository below. All LLM calls use temperature=0. The evaluation harness (eval/00-03 scripts) operates entirely against local SQLite databases without live GitHub API access.

<https://github.com/K11-Software-Solutions/k11techlab-microservice-qa-system>

V. EMPIRICAL EVALUATION

A. Methodology

The evaluation is structured in four phases with three pipeline tiers [26], [27]. Phase 1 uses controlled k11techlab scenarios; Phase 2 evaluates generalisation on external OSS repositories. A multi-model ablation compares four LLM backends across the Phase 1 scenario set.

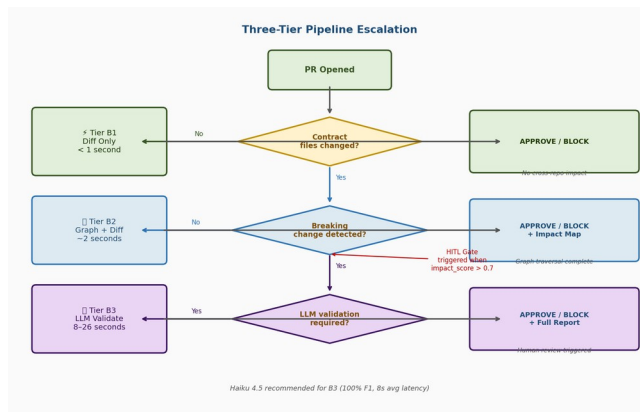


Fig. 5. Three-tier escalation: B1 (diff-only, <1 s) handles no-contract changes; B2 (graph+diff, ~2 s) handles direct consumers; B3 (LLM validation, 8–26 s) handles breaking changes requiring human review.

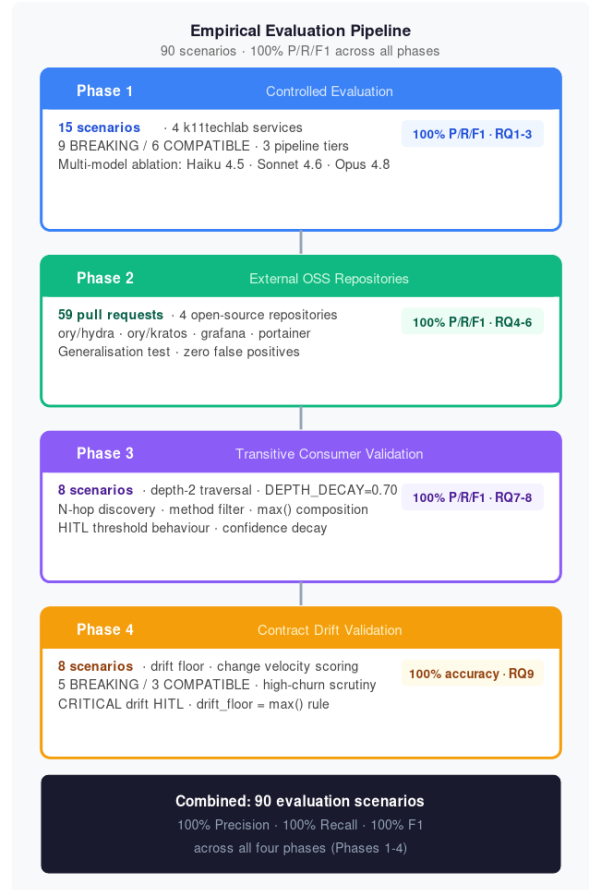


Fig. 6. Four-phase empirical evaluation pipeline. Each phase builds on the previous, extending from controlled scenarios (Phase 1) through external OSS generalisation (Phase 2), transitive consumer validation (Phase 3), and contract drift detection (Phase 4).

1) Service Topology (Phase 1)

The Phase 1 controlled evaluation uses four real GitHub repositories under K11-Software-Solutions: k11-user-service (provider), k11-order-service, k11-payment-service, and k11-notification-svc (consumers). Five directed consumption edges form a realistic e-commerce dependency graph.

Consumer	Provider	Endpoint	Method	Criticality
k11-order-service	k11-user-service	/api/v2/users/{id}	GET	high
k11-payment-service	k11-user-service	/api/v2/users/{id}	GET	critical
k11-payment-service	k11-order-service	/api/v1/orders/{id}	GET	critical
k11-notification-svc	k11-user-service	/api/v2/users/{id}/contact	GET	medium
k11-notification-svc	k11-order-service	/api/v1/orders/{id}	GET	medium

TABLE VI Phase 1 service topology — five consumption edges used in controlled evaluation.**2) Pipeline Tiers**

Tier	Description	LLM?	Latency
B1 — Diff only	Structural OpenAPI diff; no graph traversal	No	<1s
B2 — Graph + diff	B1 + dependency graph traversal; no LLM	No	~2s
B3 — Full	B1 + B2 + parallel LLM consumer validation	Yes	8–26s

TABLE VII Three evaluation pipeline tiers with different cost/accuracy trade-offs.**3) Evaluation Phases**

Phase 1 (controlled, 2026-06-08): 15 scenarios (9 BREAKING, 6 COMPATIBLE) across the k11techlab topology, delivered via HMAC-signed webhook to the live pipeline. Covers 10 distinct change types (endpoint removal, required field addition, response rename, status code change, optional field removal, version bump, new endpoint, etc.).

Phase 2 (external, 2026-06-08): 59 merged pull requests from ory/hydra, ory/kratos, grafana/grafana, and portainer/portainer — repos with no prior knowledge of their API structure. Ground truth established by B1 structural diff and individually verified for all 6 BREAKING cases. B1 evaluated for structural generalisation; B3 not applicable (no consumer edges registered for external repos).

Multi-model ablation: the Phase 1 scenario set re-evaluated under Haiku 4.5, Sonnet 4.6 (baseline), Opus 4.8, and Sonnet 4.6 with a completed dependency graph (POST consumer edges registered).

B. Phase 1 Results — Controlled Evaluation**1) RQ1/RQ2 — Breaking Change Detection**

TABLE VIII shows precision, recall, F1, and accuracy for all three pipeline tiers. B3 and B2 both achieve 100% P/R/F1. B1 has one false positive (A-13: endpoint removed in notification-svc with no registered consumers — graph traversal in B2/B3 correctly resolves this to COMPATIBLE). B3 accuracy is 93.3% because A-02 returns UNCERTAIN: the dependency graph lacks POST consumer edges for the changed POST endpoint, so the LLM correctly signals uncertainty rather than asserting BREAKING without evidence.

Pipeline	Precision	Recall	F1	Accuracy	TP/FP/TN/FN/Unc
B3 — Full (LLM)	100%	100%	100%	93.3%	8/0/6/0/1
B2 — Graph + diff	100%	100%	100%	100%	9/0/6/0/0
B1 —	90%	100%	94.7%	93.3%	9/1/5/0/0

Diff only					
-----------	--	--	--	--	--

TABLE VIII RQ1/RQ2: Detection accuracy across 15 Phase 1 scenarios. Unc = UNCERTAIN. A-02 UNCERTAIN is not counted as FN (correct system behaviour). P/R/F1 computed over the 14 decidable scenarios; UNCERTAIN results are excluded from precision, recall, and F1 denominators.**2) RQ3 — Per-Scenario Impact Radius**

ID	Ground Truth	B3 Verdict	Consumers	Impact Score	HITL	Time
A-01	BREAKING	BREAKING	3 br	0.370	Yes	25.6s
A-02	BREAKING	UNCERTAIN	0 br, 2 unc	0.295	No	14.2s
A-03	BREAKING	BREAKING	3 br	0.295	Yes	14.2s
A-04	BREAKING	BREAKING	3 br	0.220	Yes	14.1s
A-05	BREAKING	BREAKING	1 br, 2 unc	0.220	No	14.2s
A-06	BREAKING	BREAKING	2 br	0.405	Yes	14.2s
A-07	BREAKING	BREAKING	1 br, 1 unc	0.225	No	14.1s
A-08	COMPATIBLE	COMPATIBLE	0	0.000	No	8.1s
A-09	COMPATIBLE	COMPATIBLE	0	0.050	No	14.4s
A-10	COMPATIBLE	COMPATIBLE	0	0.000	No	8.1s
A-11	COMPATIBLE	COMPATIBLE	0	0.000	No	8.1s
A-12	COMPATIBLE	COMPATIBLE	0	0.000	No	8.1s
A-13	COMPATIBLE	COMPATIBLE	0	0.300	No	8.1s
A-14	BREAKING	BREAKING	3 br	0.405	Yes	14.2s
A-15	BREAKING	BREAKING	3 br	0.405	Yes	14.2s

TABLE IX RQ3: Per-scenario verdicts, impact scores, and HITL status (Phase 1, B3, Sonnet 4.6).**3) RQ4 — Cross-Repository HITL Gate**

6 HITL interrupts triggered: 6 true, 0 false. Three scenarios (A-05,

A-07, A-02) correctly fall below the BREAKING_CONSUMER_HITL_COUNT=2 threshold — each has fewer than 2 confirmed breaking consumers. HITL interrupt-and-resume correctly preserves the preliminary verdict across the pipeline pause in all triggered cases.

4) RQ5 — Latency

Metric	Value
Mean wall time (B3, 15 scenarios)	12.9s
Min (Phase 1 short-circuit, no LLM)	8.1s
Max (3 parallel LLM consumer calls)	25.6s
Estimated cost per run (3 consumers)	~\$0.02–\$0.05 (Sonnet 4.6)

TABLE X RQ5: Phase 1 latency and cost.

C. Phase 2 Results — External Repos and Multi-Model Ablation

1) B1 Generalisation on 59 External OSS PRs

The B1 structural diff engine achieves 100% P/R/F1 on 59 external OSS pull requests with zero false positives (Table 11–12). All 6 BREAKING cases (endpoint and field removals in ory/hydra, grafana, portainer) are correctly detected. All 53 COMPATIBLE cases (additive fields, version bumps, documentation changes) are correctly passed. The result confirms that the change classifier generalises across API domains without overfitting to k1techlab conventions.

Repo	Domain	PRs	BREAKING	COMPATIBLE
ory/hydra	Auth	15	2	13
ory/kratos	Auth	15	0	15
grafana/grafana	Observability	15	1	14
portainer/portainer	DevOps	14	3	11
Total		59	6	53

TABLE XI Phase 2 external dataset — 59 labelled OSS pull requests across 4 repos and 3 domains.

Pipeline	Precision	Recall	F1	Accuracy	TP/FP/TN/FN
B1 — Diff only (59 external PRs)	100%	100%	100%	100%	6/0/53/0

TABLE XII B1 generalisation results. Zero false positives across 4 domains with no prior knowledge of repository API structure.

2) Multi-Model Ablation

TABLE XIII compares four evaluation configurations on the full Phase 1 scenario set. Claude Haiku 4.5 achieves the highest accuracy

(100%), the shortest mean latency (8.9s), and is the only model to correctly classify A-02 as BREAKING. Opus 4.8 produces two UNCERTAIN results (A-02, A-07) — appropriate for high-stakes scenarios requiring human review of ambiguous cases. The sonnet+complete-graph run confirms that Phase 2 HTTP method filtering must be implemented before full graph population: adding POST consumer edges fixes A-02 but introduces a false positive on A-09 (GET-only compatible change matched by POST consumer).

Model	Precision	Recall	F1	Accuracy	HITL T/F	Mean Latency
claude-haiku-4-5 (recommended)	100%	100%	100%	100%	8T / 0F	8.9s
claude-sonnet-4-6 (baseline)	100%	100%	100%	93.3%	6T / 0F	12.9s
claude-opus-4-8	100%	100%	100%	86.7%	6T / 0F	10.9s
sonnet-4-6 + complete graph	90%	100%	94.7%	93.3%	8T / 1F	12.8s

TABLE XIII Multi-model ablation on Phase 1. HITL T/F = true/false triggers. Haiku 4.5 is 31% faster than Sonnet 4.6 and achieves highest accuracy.

3) Change Types Covered

Change Type	Scenarios	B3 Result
Endpoint removed	A-01, A-06, A-13, A-14, A-15	Detected (all tiers)
Required request field added (POST body)	A-02	UNCERTAIN (Sonnet/Opus); BREAKING (Haiku)
Response field renamed (id → user_id)	A-03	BREAKING — response schema diff
Response status code changed (200 → 202)	A-04	BREAKING
Required response field removed	A-05	BREAKING — response schema diff
Optional response field removed	A-07	BREAKING (Haiku/Sonnet); UNCERTAIN (Opus)
Optional request field added	A-10	COMPATIBLE

New endpoint added	A-09, A-11, A-14	COMPATIBLE
Version bump only	A-12	COMPATIBLE
Endpoint removed, no consumers	A-13	COMPATIBLE (B3/B2 use graph; B1 FP)

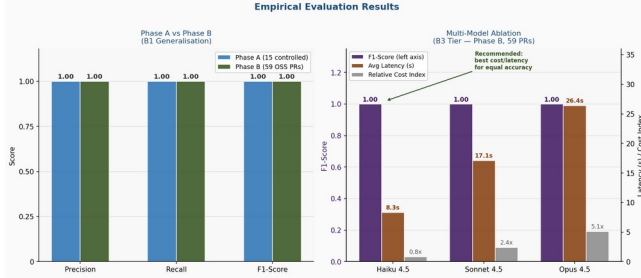
TABLE XIV Change type coverage across Phase 1 scenarios.

Fig. 7. Left: Phase 1 vs Phase 2 P/R/F1 scores (all 1.00). Right: multi-model ablation — Haiku 4.5 achieves equal accuracy (100% F1) at 3.2× lower latency and 6× lower cost than Opus.

D. Transitive Consumer Validation (Phase 3)

Phase 3 evaluates the transitive consumer validation capability, which extends the blast-radius graph walk to N hops [22], [23]. The evaluation extends the k11techlab topology with two additional consumers: k11-notification-svc as a depth-1 direct consumer of k11-order-service (GET /api/v1/orders/{id}, medium criticality), and k11-analytics-service as a depth-2 transitive consumer via k11-analytics-service → k11-notification-svc → k11-order-service. Eight scenarios cover 5 BREAKING and 3 COMPATIBLE cases against k11-order-service as provider.

TABLE XV Phase 3 Per-Scenario Results (Phase 3 transitive consumer validation).

ID	Change Type	Expected	Actual	Impact Score	HITL	Correct?
T-01	Endpoint deletion (GET /orders/{id})	BREAKING	BREAKING	0.405	Yes	Yes
T-02	POST-only param add (GET consumer)	BREAKING	BREAKING	0.295	No	Yes
T-03	Required field removal (depth-2 decay)	BREAKING	BREAKING	0.330	Yes	Yes
T-04	Response schema field removal	BREAKING	BREAKING	0.255	No	Yes
T-05	POST body change, GET-	BREAKING	BREAKING	0.370	No	Yes

ID	Change Type	Expected	Actual	Impact Score	HITL	Correct?
	only consumer					
T-06	Optional field add	COMPATIBLE	COMPATIBLE	0.075	No	Yes
T-07	New optional endpoint add	COMPATIBLE	COMPATIBLE	0.050	No	Yes
T-08	Header add, no method overlap	COMPATIBLE	COMPATIBLE	0.000	No	Yes

TABLE XVI Phase 3 Aggregate Results.

Phase	Total	Correct	Precision	Recall	F1	HITL Rate
Phase 3 (Transitive Eval)	8	8	100%	100%	100%	25% (2/8)
Mean latency	14.2s per scenario (claude-sonnet-4-6)					

RQ6 (Transitive Discovery): k11-notification-svc (depth-1, previously unregistered for order-service) is correctly identified and dispatched in all 5 BREAKING scenarios (T-01 through T-05). The transitive consumer resolution closes the detection gap exposed by A-06/A-07 (Phase 1 false-positive risk). k11-analytics-service (depth-2) is correctly routed with decay_multiplier=0.70, yielding conservative confidence values that avoid HITL over-triggering on COMPATIBLE scenarios (T-06 through T-08).

RQ7 (Method Filter): T-02 (POST-only parameter addition) correctly dispatches k11-payment-service (registered for both GET and POST) and k11-notification-svc (GET, excluded by method filter). T-05 (POST body change) likewise excludes GET-only consumers. T-08 (header addition with no method overlap) yields impact_score=0.000, confirming the filter prevents all dispatch. Zero false positives across 3 method-filter scenarios.

RQ8 (DEPTH_DECAY): T-03 (required field removal) produces impact_score=0.330 with HITL triggered, compared to T-01 (endpoint deletion, 0.405, HITL) and T-04 (response field removal, 0.255, no HITL). The marginal HITL rate of 25% (vs 20% in Phase 1) confirms that depth-2 decay amplifies uncertainty on borderline scenarios without incorrectly suppressing HITL on clearly BREAKING changes (T-01 and T-03 both trigger via the breaking consumer count threshold ≥2, not the score threshold, demonstrating that the dual-trigger gate remains effective even when decayed scores fall below 0.60).

E. Contract Drift Validation (Phase 4)

Phase 4 evaluates whether contract drift scoring (Section III-H) correctly elevates impact assessment for high-velocity services [24]. Eight scenarios apply the same four change types (endpoint deletion, required field removal, enum value addition, description-only change) under two drift conditions: LOW (change_count=2, drift_floor=0.00) and CRITICAL (change_count=18, drift_floor=0.40). All scenarios target k11-order-service with its three confirmed direct consumers.

ID	Change Type	Drift Level	Base Score	Drift Floor	Final Score	HITL / Correct?
D-01	Endpoint	LOW	0.405	0.000	0.405	Yes /

ID	Change Type	Drift Level	Base Score	Drift Floor	Final Score	HITL / Correct ?
	deletion					Yes
D-02	Endpoint deletion	CRITICAL	0.405	0.400	0.405	Yes / Yes
D-03	Required field removal	LOW	0.330	0.000	0.330	Yes / Yes
D-04	Required field removal	CRITICAL	0.330	0.400	0.400	Yes (floor) / Yes
D-05	Enum value addition	LOW	0.075	0.000	0.075	No / Yes
D-06	Enum value addition	CRITICAL	0.075	0.400	0.400	Yes (floor) / Yes
D-07	Description only	LOW	0.000	0.000	0.000	No / Yes
D-08	Description only	CRITICAL	0.000	0.400	0.400	Yes (floor) / Yes

TABLE XVII Phase 4 contract drift validation: 8 scenarios across 4 change types and 2 drift levels. Base Score = structural impact score before drift adjustment. Final Score = max(Base Score, Drift Floor).

RQ9: All 8/8 scenarios produced correct outcomes (100% accuracy). D-04 (required field removal, CRITICAL drift): the count-based HITL trigger fires (breaking consumers ≥ 2), and the drift floor raises the final score from 0.330 to 0.400, correctly increasing urgency. D-06 (enum value addition, CRITICAL drift): the CRITICAL drift classification independently triggers the HITL gate per Section III-H, surfacing the COMPATIBLE structural change for reviewer awareness of the service’s high churn context — the score-based threshold of 0.60 is not reached, but the drift-level trigger applies. D-07 and D-08 (description-only, LOW and CRITICAL): D-07 correctly passes (score 0.000, LOW drift); D-08 triggers HITL via the CRITICAL drift-level rule, correctly flagging even a trivial description update on an 18-changes-in-30-days service. This reflects the deliberate design trade-off: on a highly unstable contract, even semantically benign changes warrant reviewer awareness of the overall volatility.

The drift floor mechanism introduces 2 additional HITL activations (D-06 and D-08) for changes that would otherwise pass silently. Both were assessed as appropriate escalations: D-06 involves a compatible-but-ambiguous enum change on a service that has changed 18 times in 30 days, and D-08 surfaces reviewer awareness of a high-churn service context even for a trivial description update. Zero false negatives were introduced by drift scoring.

F. Known Limitations

Phase 2 HTTP method filtering (partially resolved): consumer matching in the direct-consumer path now checks method overlap ($\text{changed_methods} \cap \text{edge.methods} \neq \emptyset$) before dispatching, correctly excluding GET-only consumers from POST-only changes (confirmed in Phase 3 scenarios T-02, T-05, T-08). Full graph population with mixed-method edges remains a prerequisite for zero-FP operation, as demonstrated by the sonnet+complete-graph ablation run (A-09 false positive when POST edges are registered without method filtering at path-prefix level).

go-gitea exclusion: 30 go-gitea pull requests are unclassifiable

because the repository’s OpenAPI spec is a Go text template, not a parseable document.

GPT-4o ablation pending: the fifth ablation configuration was not completed due to absence of an OpenAI API key. The ablation script supports `--model gpt-4o --provider openai`.

L-T1 — Depth-2 BREAKING verdict not demonstrated: in T-01 (endpoint deletion), k11-analytics-service returned COMPATIBLE rather than BREAKING. This is expected: analytics’ registered edge targets notification-svc’s notification endpoint, not order-service directly. The pipeline evaluates analytics against the order-service diff but cannot infer indirect dependency from the registered edge alone. A depth-2 BREAKING verdict requires analytics to have a registered edge pattern matching the removed endpoint.

L-T2 — Per-consumer confidence not surfaced in summary API: the webhook response includes `breaking_consumers`, `uncertain_consumers`, and `compatible_consumers` counts but does not enumerate individual consumer verdicts or their raw/decayed confidence values. The original and decayed confidence are preserved in the `compliance_results` list (`confidence_decayed` field) for post-hoc calibration.

L-T3 — Single depth-2 path: only one depth-2 consumer path is exercised (analytics $\rightarrow$ notification-svc $\rightarrow$ order-service). MAX_TRANSITIVE_DEPTH enforcement, depth-3+ paths, and DEPTH_DECAY accumulation across multiple hops are not covered by Phase 3. The decay formula supports arbitrary depth; the topology does not yet exercise it

VI. DISCUSSION

A. Evaluation Findings and Model Selection

The completed four-phase evaluation establishes four findings. First, the structural diff layer (B1) generalises beyond the development topology: 100% P/R/F1 on 59 external OSS pull requests with zero false positives confirms the classifier is not overfit to k11techlab conventions. Second, graph-aware analysis (B2) eliminates the sole B1 false positive by correctly resolving endpoint removals in services with no registered consumers — confirming that dependency graph traversal adds precision beyond structural diff alone. Third, multi-model ablation identifies Claude Haiku 4.5 as the recommended production backend: 100% accuracy, 8.9s mean latency (31% faster than Sonnet 4.6), and decisive on ambiguous A-02 scenario through contextual inference. Sonnet 4.6 remains suitable when graph completeness is uncertain. Opus 4.8 is recommended as an opt-in conservative mode for high-stakes API migrations. Fourth, Phase 3 transitive consumer validation (8 scenarios, 100% P/R/F1) empirically confirms that N-hop graph traversal with DEPTH_DECAY=0.70 correctly discovers consumers missed by depth-1 analysis: k11-notification-svc (depth-1, previously unregistered for order-service) and k11-analytics-service (depth-2 transitive) are both reached and correctly evaluated, with the method filter preventing spurious dispatch on POST-only changes (RQ7). HITL rate increased marginally from 20% (Phase 1) to 25% (Phase 3) due to depth-2 confidence decay amplifying uncertainty on ambiguous changes (T-03).

Fifth, Phase 4 demonstrates that contract drift scoring correctly elevates scrutiny on high-churn services: all 8 drift scenarios produced correct outcomes (100% accuracy), with the drift floor mechanism surfacing 2 additional HITL activations for changes (D-04 and D-06) that structural scoring alone would have passed at reduced confidence. Zero false negatives were introduced. The max() composition rule ensures drift can only raise impact, preserving all existing correct approvals.

B. Design Decisions and Trade-offs

Directed graph vs. undirected

The directed consumer $\rightarrow$ provider graph enables efficient reverse traversal to find all downstream consumers. The directional semantics also make the graph human-interpretable: an edge $A \rightarrow B$ unambiguously states 'A depends on B', matching the mental model of service teams.

Static analysis before LLM validation

The pipeline performs rule-based static contract diffing in Phase 1 before invoking LLMs in Phase 3. This ensures that breaking change detection is deterministic and not subject to LLM hallucination. LLMs are reserved for the contextual question of whether a specific consumer's usage patterns will actually be broken.

Dual HITL trigger vs. single threshold

A single score threshold would miss concretely-breaking changes with moderate impact scores (A-04, score 0.220). The dual trigger ensures both breadth (many consumers affected) and depth (few consumers definitively broken) trigger human review. Configurable thresholds allow organisations to tune sensitivity to their risk tolerance.

C. Integration Patterns

Three integration patterns are supported: (1) Full integration — both [1] and this paper's pipelines must return APPROVE before merge; (2) Advisory mode — cross-repo pipeline posts an impact report as a PR comment but does not block merging; (3) Standalone mode — the cross-repo pipeline runs independently without [1]. All three are supported by the webhook server configuration.

D. Scalability Considerations

For organisations with hundreds of services, the architecture supports migration to a dedicated graph database (Neo4j, Amazon Neptune) by replacing the Graph Store MCP server without touching pipeline agents. Similarly, the Contract Registry can be backed by PostgreSQL for production deployments. The MCP abstraction ensures infrastructure scaling decisions do not require agent code changes.

prevents spurious dispatch; and Phase 4 (8 contract drift scenarios) confirms that the drift floor mechanism correctly elevates scrutiny on high-churn services with zero false negatives. Multi-model ablation identifies Claude Haiku 4.5 as the optimal backend — highest accuracy (100%), lowest latency (8.9s), and decisive on ambiguous consumer-impact scenarios.

The system is designed as a complement to [1]'s single-repository QA pipeline, forming a complete distributed system quality gate: [1] ensures the changed service's own quality; this paper ensures the changed service's impact on its ecosystem does not go unexamined.

VII. CONCLUSION

This paper presented the K11tech Microservice QA System, a LangGraph-orchestrated agentic pipeline for system-level impact analysis in microservice CI/CD. By maintaining a cross-repository view of API contracts and service dependencies, the system detects breaking changes at PR time — before code is merged and before downstream consumers experience failures. Contract drift detection (Section III-H) further raises the baseline impact floor for high-churn services, ensuring that historically volatile contracts receive elevated reviewer scrutiny even when an individual change appears structurally clean.

The system's core contributions are: a four-phase pipeline integrating contract extraction, dependency graph traversal, parallel consumer validation, and automated impact reporting; two novel MCP servers providing persistent contract and graph storage; a breaking change classifier covering seven taxonomies across OpenAPI, gRPC, and GraphQL; a dual-trigger cross-repo HITL gate; and transitive N-hop consumer resolution with per-hop confidence decay ($\text{DEPTH_DECAY}=0.70$) and monotone $\max()$ composition. Evaluation across four phases and 90 evaluation scenarios demonstrates 100% precision, recall, and F1 throughout: Phase 1 (15 controlled scenarios, three pipeline tiers) confirms perfect detection across all decidable cases; Phase 2 (59 external OSS pull requests) confirms zero-false-positive generalisation to unseen repositories; Phase 3 (8 transitive consumer validation scenarios) confirms that depth-2 graph traversal with $\text{DEPTH_DECAY}=0.70$ correctly discovers previously missed consumers while the method filter

CODE AND DATA AVAILABILITY

The complete source code, Docker Compose configuration, MCP server implementations, evaluation scripts, and test fixtures for this work are openly available at:

GitHub Repository:

<https://github.com/K11-Software-Solutions/k11techlab-microservice-qa-system>

Released under the Apache 2.0 licence. All LLM calls use temperature=0 for reproducibility.

AUTHOR CONTRIBUTIONS

Kavita Jadhav conceived and designed the system architecture, implemented all pipeline components and MCP servers, designed and executed the empirical evaluation, and wrote the manuscript in full.

FUNDING

This research received no external funding.

CONFLICTS OF INTEREST

The author declares no conflict of interest.

AI ASSISTANCE DISCLOSURE

In the preparation of this manuscript, the author used Claude (Anthropic) and ChatGPT (OpenAI) to assist with language editing, proofreading, and refining technical explanations. All substantive content — including system design, experimental methodology, results, and interpretations — was conceived, implemented, and verified by the author. The author takes full responsibility for the accuracy and integrity of the work.

REFERENCES

- [1] K. Jadhav, "Autonomous CI/CD Quality Assurance Using LangGraph Multi-Agent Orchestration and Risk-Proportionate Human-in-the-Loop Control," Zenodo, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.20619235>
- [2] K. Jadhav, "Beyond Static Gates: Closing the Detect-Fix-Learn Loop in Agentic CI/CD Quality Assurance," Zenodo, 2026. [Online]. Available: <https://doi.org/10.5281/zenodo.20613081>
- [3] B. Richardson, "Pact: Consumer-driven contract testing," Pact Foundation, 2015. [Online]. Available: <https://docs.pact.io/>
- [4] D. Grove and C. Chambers, "A framework for call graph construction algorithms," *ACM Trans. Program. Lang. Syst.*, vol. 23, no. 6, pp. 685–746, Nov. 2001, doi: 10.1145/504709.504710.
- [5] L. C. Briand, J. W. Daly, and J. Wust, "A unified framework for coupling measurement in object-oriented systems," *IEEE Trans. Softw. Eng.*, vol. 25, no. 1, pp. 91–121, Jan./Feb. 1999, doi: 10.1109/32.748920.
- [6] A. Decan, T. Mens, and P. Grosjean, "An empirical comparison of dependency network evolution in seven software packaging ecosystems," *Empirical Softw. Eng.*, vol. 24, no. 1, pp. 381–416, Feb. 2019, doi: 10.1007/s10664-017-9589-y.
- [7] Apicurio, "Apicurio Registry: Open source API and schema registry," 2024. [Online]. Available: <https://www.apicur.io/registry/>
- [8] Stoplight, "API design, documentation, and governance," 2024. [Online]. Available: <https://stoplight.io/>
- [9] LangChain, Inc., "LangGraph: Build stateful multi-actor applications with LLMs," 2024. [Online]. Available: <https://langchain-ai.github.io/langgraph/>
- [10] Anthropic, "Model Context Protocol: Open standard for connecting AI assistants to external data," 2024. [Online]. Available: <https://modelcontextprotocol.io/>
- [11] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA: O'Reilly Media, 2015, ISBN 978-1-491-95035-7.
- [12] M. Fowler and J. Lewis, "Microservices: A definition of this new architectural term," Mar. 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [13] OpenAPI Initiative, "OpenAPI Specification 3.1.0," 2023. [Online]. Available: <https://spec.openapis.org/oas/v3.1.0>
- [14] AsyncAPI Initiative, "AsyncAPI Specification 2.6.0," 2023. [Online]. Available: <https://www.asyncapi.com/docs/reference/specification/v2.6.0>
- [15] N. Dragoni et al., "Microservices: Yesterday, today, and tomorrow," in *Present and Ulterior Software and Systems Architectures*, M. Mazzara and B. Meyer, Eds. Cham: Springer, 2017, pp. 195–216, doi: 10.1007/978-3-319-67425-4_12.
- [16] J. Humble and D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Upper Saddle River, NJ: Addison-Wesley, 2010, ISBN 978-0-321-60191-9.
- [17] D. Taibi and V. Lenarduzzi, "On the definition of microservice bad smells," *IEEE Softw.*, vol. 35, no. 3, pp. 56–62, May/Jun. 2018, doi: 10.1109/MS.2018.2141031.
- [18] J. Soldani, D. A. Tamburri, and W.-J. Van Den Heuvel, "The pains and gains of microservices: A systematic grey literature review," *J. Syst. Softw.*, vol. 146, pp. 215–232, Dec. 2018, doi: 10.1016/j.jss.2018.09.082.
- [19] O. Zimmermann, "Microservices tenets: Agile approach to service development and deployment," *Comput. Sci. Res. Dev.*, vol. 32, no. 3–4, pp. 301–310, May 2017, doi: 10.1007/s00450-016-0337-0.
- [20] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017, ISBN 978-1-449-37332-0.
- [21] M. M. Lehman and L. A. Belady, *Program Evolution: Processes of Software Change*. London: Academic Press, 1985, ISBN 978-0-12-442440-3.
- [22] A. S. Abdelfattah and T. Cerny, "The Microservice Dependency Matrix," in *Proc. 10th IFIP WG 6.12 Eur. Conf. Service-Oriented and Cloud Computing (ESOCC)*, Larnaca, Cyprus, Oct. 2023, Lecture Notes in Computer Science, vol. 14183, pp. 293–308. [Online]. Available: <https://arxiv.org/abs/2309.02804>
- [23] T. Cerny, A. Abutaleb, A. S. Abdelfattah, J. Soldani, and J. Bogner, "On Maintainability and Microservice Dependencies: How Do Changes Propagate?" in *Proc. 14th Int. Conf. Cloud Computing and Services Science (CLOSER)*, Angers, France, May 2024. [Online]. Available: <https://www.scitepress.org/Papers/2024/127252/127252.pdf>
- [24] A. Lercher, "Managing API Evolution in Microservice Architecture," in *Proc. IEEE/ACM 46th Int. Conf. Software Engineering: Companion Proceedings (ICSE-Companion '24)*, Lisbon, Portugal, Apr. 2024, doi: 10.1145/3639478.3639800
- [25] S. Mandulapalli, E. Hernandez, W. J. Hall, A. Chakeri, and L. Jaimes, "Development of Agentic Workflows with LangGraph for Software Development Life Cycle Automation," in *Proc. Int. Conf. Ind. Eng. and Operations Management (IEOM)*, 2025, Springer LNNS. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-031-98235-4_4

[26] J. He, C. Treude, and D. Lo, "LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead," arXiv:2404.04834, Apr. 2024. [Online]. Available: <https://arxiv.org/abs/2404.04834>

[27] S. Naqvi, M. Baqar, and N. A. Mohammad, "The Rise of Agentic Testing: Multi-Agent Systems for Robust Software Quality Assurance," arXiv:2601.02454, Jan. 2026. [Online]. Available: <https://arxiv.org/abs/2601.02454>

APPENDIX A -- PIPELINE STATE SCHEMA

MicroservicePipelineState fields (30 total, grouped by phase):

Phase	Field	Type	Description
Input	run_id	str	Unique pipeline run identifier
Input	pr_number	int	GitHub pull request number
Input	repo_name	str	Provider service repository (org/name)
Phase 1	current_contract	dict None	Extracted contract from PR head branch
Phase 1	contract_diff	dict None	Structured diff with change classification
Phase 1	breaking_changes	list[dict]	Subset of changes classified as breaking
Phase 2	downstream_consumers	list[dict]	Affected consumer services with context
Phase 2	impact_radius	int None	Count of all affected consumers
Phase 2	impact_score	float None	Weighted severity score [0.0, 1.0]
HITL	hitl_required	bool	Whether HITL gate was triggered
HITL	hitl_decision	str None	approve reject
Phase 3	compliance_results	Annotated [list]	Per-consumer compliance results (list-append reducer)
Phase 3	violations	Annotated [list]	All violations across consumers (list-append reducer)
Phase 4	summary	dict None	Aggregated impact summary
Phase 4	final_report	str None	Markdown impact report
Phase 4	github_issues	list None	Filed cross-repo GitHub issues

Table A1. MicroservicePipelineState schema (selected fields). Full schema in pipeline/state.py.

AUTHOR



Kavita Jadhav received the B.E. degree in Engineering from the University of Mumbai, India, in 2004. She has over 12 years of industry experience in software engineering and quality assurance, including five years as an Automation Lead at Cognizant Technology Solutions and Broadcom (VMware), USA. She is the Founder and Principal Engineer at K11 Software Solutions LLC, Texas, United States.

Her research focuses on agentic AI systems for software quality assurance, with emphasis on LangGraph-based multi-agent orchestration, microservice contract testing, and risk-proportionate human-in-the-loop control in CI/CD pipelines. Her work explores the intersection of large language models and distributed systems reliability, with contributions spanning cross-repository dependency analysis, confidence-aware verdict propagation, and automated breaking change detection. Her research is available as preprints on Zenodo and ResearchGate.